

优极限

“极限教育，挑战极限”

www.yjxxt.com

极限教育，挑战极限。优极限是一个让 95% 的学生年薪过 18 万的岗前培训公司，让我们的学员具备优秀的互联网技术和职业素养，勇攀高薪，挑战极限。公司位于上海浦东，拥有两大校区，共万余平。累计培训学员超 3 万名。我们的训练营就业平均月薪 19000，最高年薪 50 万。

核心理念：让学员学会学习，拥有解决问题的能力，拿到高薪职场的钥匙。

项目驱动式团队协作、一对一服务、前瞻性思维、教练式培养模型-培养你成为就业明星。首创的老学员项目联盟给学员充分的项目、技术支撑，利用优极限平台这根杠杆，不断挑战极限，勇攀高薪，开挂人生。

扫码关注优极限微信公众号：

（获取最新技术相关资讯及更多源码笔记）



SpringBoot - 第一天

1. 学习目标



2. Spring框架发展史

2.1. Spring1.x 时代

在 Spring1.x 时代，都是通过 xml 文件配置 bean，随着项目的不断扩大，需要将 xml 配置分放到不同的配置文件中，需要频繁的在 java 类和 xml 配置文件中切换。

2.2. Spring2.x 时代

随着 JDK 1.5 带来的注解支持，Spring2.x 可以使用注解对 Bean 进行声明和注入，大大的减少了 xml 配置文件，同时也大大简化了项目的开发。

那么，问题来了，究竟是应该使用 xml 还是注解呢？

最佳实践：

1. 应用的基本配置用 xml，比如：数据源、资源文件等；
2. 业务开发用注解，比如：Service 中注入 bean 等；

2.3. Spring3.x 到 Spring4.x 再到 Spring5.x

从 Spring3.x 开始提供了 Java 配置方式，使用 Java 配置方式可以更好的理解你配置的 Bean，现在我们就处于这个时代，并且 Spring4.x、Spring5.x 和 Spring Boot 都推荐使用 java 配置的方式。

3. Spring 5.X 应用零配置开发

Spring 框架从 5.x 版本推荐使用注解形式来对 java 应用程序进行开发与配置，并且可以完全替代原始的 XML + 注解形式的开发，在使用注解形式进行项目开发与环境配置时，Spring 框架提供了针对环境配置与业务 bean 开发相关注解。

3.1. 注解

3.1.1. 声明Bean 注解

@Component: 组件 没有明确规定其角色，作用在类级别上声明当前类为一个业务组件，被Spring Ioc 容器维护

@Service: 在业务逻辑层（Service 层）类级别进行声明

@Repository: 在数据访问层(dao 层) 类级别声明

@Controller: 在展现层(MVC) 使用 标注当前类为一个控制器

3.1.2. 注入Bean 注解

@Autowired: Spring 官方提供注解

@Inject: JSR-330 提供注解（标准制定方）

@Resource: JSR-250 提供注解

以上三种注解在 Set 方法或属性上声明，一般情况下通用一般开发中更习惯声明在属性上，代码简洁清晰。基于5.x 注解配置方式简化了xml 配置，应用程序开发与xml 环境配置均通过相应注解来实现。

3.1.3. Spring5.x 中配置与获取Bean注解

@Configuration: 作用与类上，将当前类声明为一个配置类，相当于一个xml 配置文件

@ComponentScan: 自动扫描指定包下标注有@Repository,@Service,@Controller

@Component: 注解的类并由Ioc 容器进行实例化和维护

@Bean: 作用于方法上,相当于xml 文件中<bean> 声明当前方法返回值为一个bean

@Value: 获取properties 文件指定key value值

3.2. 实例1 - IOC 中 Bean 的实例化与获取

3.2.1. 创建 Spring 普通工程

在 pom.xml 中添加坐标相关配置

```
<dependencies>
  <!-- spring的依赖坐标 -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.2.4.RELEASE</version>
  </dependency>
</dependencies>

<build>
  <plugins>
```

```

<!-- 指定Maven编译的JDK版本和编码 -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>2.3.2</version>
  <configuration>
    <!-- 源代码使用的JDK版本 -->
    <source>11</source>
    <!-- 需要生成的目标class文件的编译版本 -->
    <target>11</target>
    <!-- 字符集编码 -->
    <encoding>utf-8</encoding>
  </configuration>
</plugin>
</plugins>
</build>

```

3.2.2. 创建 Bean 对象

UserDao.java

```

@Repository
public class UserDao {
    public void test(){
        System.out.println("UserDao.test...");
    }
}

```

UserService.java

```

@Service
public class UserService {
    @Resource
    private UserDao userDao;

    public void test(){
        System.out.println("UserService.test...");
        userDao.test();
    }
}

```

3.2.3. 创建 IocConfig 配置类

```

// 将当前类声明为一个配置类
@Configuration
// 设置扫描包范围
@ComponentScan("com.xxxx.springboot")
public class IocConfig {

}

```

3.2.4. 创建启动类执行测试

```
public class Starter {
    public static void main(String[] args) {
        // 基于Java的配置类加载Spring的应用上下文
        AnnotationConfigApplicationContext ac = new
        AnnotationConfigApplicationContext(IocConfig.class);
        // 获取指定的Bean对象
        UserService userService = ac.getBean(UserService.class);
        // 调用Bean对象的方法
        userService.test();
    }
}
```

此时启动 Spring IOC 容器，通过实例化 **AnnotationConfigApplicationContext** 类，接收配置参数类 **IocConfig**，并获取 **UserService** Bean 实现方法调用，此时应用环境不存在 xml 配置文件，简化了应用的 xml 配置。

3.3. 实例2 - @Bean注解使用

使用 **@Bean** 注解声明在方法（注意：方法名一般为bean对象名称）级别用于返回实例化的Bean对象。

3.3.1. 创建 Bean 对象

AccountDao.java

```
// 注意：此时类级别并未添加 @Repository 注解
public class AccountDao {
    public void test(){
        System.out.println("AccountDao.test...");
    }
}
```

3.3.2. 修改 IocConfig 配置类

添加返回AccountDao Bean对象方法

```
@Configuration
@ComponentScan("com.xxxx.springboot")
public class IocConfig02 {

    // 返回实例化的单例Bean对象
    @Bean
    public AccountDao accountDao(){
        return new AccountDao();
    }
}
```

3.3.3. 创建启动类并执行测试

```
public class Starter02 {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext ac = new
        AnnotationConfigApplicationContext(IocConfig.class);
        // 判断IocConfig对象是否是单例
        System.out.println(ac.isSingleton("IocConfig"));
        // 获取IocConfig对象
        IocConfig iocConfig = ac.getBean(IocConfig.class);

        // 获取AccountDao对象
        AccountDao accountDao01 = iocConfig.accountDao();
        AccountDao accountDao02 = iocConfig.accountDao();
        System.out.println(accountDao01 + ", " + accountDao02);

        accountDao01.test();
    }
}
```

3.4. 实例3 - 读取外部配置文件

在开发 Java web 应用时，配置文件是比较常见的，比如 xml，properties，yaml 等文件，在 Spring 应用中对于配置文件的读取同样提供支持。对于配置文件读取，我们可以通过 @PropertySource 注解声明到类级别来指定读取相关配置。

Spring EL 表达式语言，支持在 Xml 和注解中使用表达式，类似于 JSP 中 EL 表达式，Spring 框架借助该表达式实现资源注入，主要通过 @Value 注解来使用表达式，通过 @Value 注解，可以实现普通字符串，表达式运算结果，Bean 属性文件内容，属性文件等参数注入。具体使用如下：

3.4.1. 准备配置文件

src/main/resources 目录下添加 user.properties、jdbc.properties 文件

```
# user.properties
user.userName=admin
user.password=admin

# jdbc.properties
jdbc.driver=com.mysql.jdbc.Driver
jdbc.url=jdbc:mysql://127.0.0.1:3306/hr?useUnicode=true&characterEncoding=utf8
jdbc.username=root
jdbc.password=root
```

3.4.2. @PropertySource 加载配置文件

通过 @PropertySource 加载 properties 配置文件

```
@Configuration
@ComponentScan("com.xxx")
@PropertySource(value =
{"classpath:jdbc.properties", "classpath:user.properties"})
public class IocConfig03 {
```

```

@Value("${jdbc.driver}")
private String driver;
@Value("${jdbc.url}")
private String url;
@Value("${jdbc.username}")
private String userName;
@Value("${jdbc.password}")
private String password;

// 控制台打印属性值信息
public void showConfigInfo(){
    System.out.println("driver:" + driver + ", url:" + url);
    System.out.println("userName:" + userName + ", password:" + password);
}
}

```

3.4.3. 其他Bean对象获取properties文件内容

```

@Service
public class UserService {
    @Resource
    private UserDao userDao;

    @Value("${user.userName}")
    private String userName;
    @Value("${user.password}")
    private String password;

    public void test(){
        System.out.println("UserService.test...");
        userDao.test();

        System.out.println("userName:" + userName + ", password:" + password);
    }
}

```

启动，查看控制台输出内容效果

3.5. 组合注解与元注解

Spring 从2.x版本开始引入注解支持(目的是jdk1.5中推出注解功能)，通过引入注解来消除大量xml 配置，Spring 引入注解主要用来注入 bean 以及 aop 切面相关配置，但由于注解大量使用，就会造成大量重复注解代码出现，代码出现了重复，Spring 为了消除重复注解，在元注解上引入了组合注解，其实可以理解是对代码的重构，相当于注解的注解，拥有元注解的原始功能，比如在定义配置类时用到的 @Configuration 注解就是组合注解，拥有 @Component 注解功能，即配置类本身也是一个被 IOC 维护的单例 Bean。

```

@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Component
public @interface Configuration {

```

3.5.1. 自定义组合注解

- 定义 MyCompScan 注解，拥有 @ComponentScan 扫描器注解功能

```

/**
 * 组合注解MyCompScan 定义
 * 拥有元注解@Configuration + @ComponentScan 两者功能
 * 覆盖 value 属性
 */
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Configuration
@ComponentScan
public @interface MyCompScan {
    //设置别名，当前属性为哪个注解服务
    @AliasFor(annotation = ComponentScan.class, value = "value")
    String[] value() default {};
}

```

3.5.2. 应用组合注解

```

@MyCompScan("com.xxxx.springboot")
@PropertySource(value =
{"classpath:jdbc.properties","classpath:user.properties"})
public class IocConfig04 {

    @Value("${jdbc.driver}")
    private String driver;
    @Value("${jdbc.url}")
    private String url;
    @Value("${jdbc.username}")
    private String userName;
    @Value("${jdbc.password}")
    private String password;

    public void showConfigInfo(){
        System.out.println("driver:" + driver + ", url:" + url);
        System.out.println("userName:" + userName + ", password:" + password);
    }
}

```

测试组合注解

```

public class Starter {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext ac = new
        AnnotationConfigApplicationContext(IocConfig04.class);
        UserService userService = ac.getBean(UserService.class);
        userService.test();
    }
}

```

4. Spring MVC 零配置创建与部署

基于 Spring Mvc 5.X 使用 Maven 搭建 SpringMvc Web项目，通过 Spring 提供的注解与相关配置来对项目进行创建与部署。

使用tomcat 运行此项目，访问测试

4.1. 创建Spring MVC Web工程

创建 Maven 的 web 项目

4.2. pom.xml 添加坐标相关配置

```

<!-- spring web -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-web</artifactId>
    <version>5.2.4.RELEASE</version>
</dependency>

<!-- spring mvc -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.2.4.RELEASE</version>
</dependency>

<!-- web servlet -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.0.1</version>
    <scope>provided</scope>
</dependency>

<build>
    <finalName>springmvc</finalName>
    <plugins>
        <!-- 指定Maven编译的JDK版本和编码 -->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>2.3.2</version>
            <configuration>
                <source>11</source>
            </configuration>
        </plugin>
    </plugins>
</build>

```

```

        <target>11</target>
        <encoding>utf-8</encoding>
    </configuration>
</plugin>
</plugins>
</build>

```

4.3. 添加源代码

```

@Controller
public class HelloController {
    @RequestMapping("/index")
    public String index(){
        return "index";
    }
}

```

4.4. 添加视图

在 WEB-INF/views 目录下创建 index.jsp (这里以jsp为模板)

```

<html>
    <body>
        <h2>Hello mvc! </h2>
    </body>
</html>

```

4.5. SpringMVC 配置类添加

Spring Mvc 配置信息 MvcConfig 文件添加，作为 Mvc 框架环境，原来是通过 xml 来进行配置 (视图解析器、Json转换器、文件上传解析器等)，这里基于注解通过继承 WebMvcConfigurerAdapter 类并重写相关方法来进行配置 (注意通过 @EnableWebMvc 注解来启动 MVC 环境)。

```

/**
 * MVC 基本配置
 */
// 配置类
@Configuration
// 在@Configuration注解的配置类中添加，用于为该应用添加SpringMVC的功能
@EnableWebMvc
// 扫描包范围
@ComponentScan("com.xxxx.springboot")
public class MvcConfig {

    /**
     * 配置 JSP视图解析器
     * @return
     */
    @Bean // 将方法返回的结果交给IOC容器维护
    public InternalResourceViewResolver viewResolver(){
        // 获取视图解析器
        InternalResourceViewResolver viewResolver = new
        InternalResourceViewResolver();
        // 设置前缀
        viewResolver.setPrefix("/WEB-INF/views/");
    }
}

```

```

        // 设置后缀
        viewResolver.setSuffix(".jsp");
        // 返回解析器对象 （交给IOC容器进行维护）
        return viewResolver;
    }
}

```

MvcConfig 类定义好了，那么问题来了，怎么加载 MvcConfig 类呢，原来在构建 Mvc 应用时是通过容器启动应用时加载 web.xml 文件实现配置文件加载，现在的环境 web.xml 文件不存在，此时基于注解方式构建的 Mvc 应用，定义 WebInitializer 实现 WebApplicationInitializer 接口 (该接口用来配置 Servlet3.0+ 配置的接口，用于替代 web.xml 配置)，当 servlet 容器启动Mvc应用时会通过 SpringServletContainerInitializer 接口进行加载，从而加载 Mvc 应用信息配置。实现该接口 onStartUp 方法，加载应用信息配置。

4.6. 入口文件代码添加

```

/**
 * 实现 webApplicationInitializer 接口的类都可以在web应用程序启动时被加载
 */
public class WebInitializer implements WebApplicationInitializer {
    @Override
    public void onStartUp(ServletContext servletContext) throws ServletException
    {
        // 基于Java的配置类加载Spring的应用上下文
        AnnotationConfigWebApplicationContext ctx = new
        AnnotationConfigWebApplicationContext();
        // 注册 MVC 配置信息
        ctx.register(MvcConfig.class);
        // 设置 ServletContext 上下文信息
        ctx.setServletContext(servletContext);
        // 配置转发器 Dispatcher
        ServletRegistration.Dynamic servlet =
        servletContext.addServlet("dispatcher", new DispatcherServlet(ctx));
        // 设置映射路径
        servlet.addMapping("/");
        // 启动时即实例化 Bean
        servlet.setLoadOnStartup(1);
    }
}

```

4.7. 部署与测试

通过 Tomcat 启动项目并访问

← → ↻ 🌐 localhost:8080/mvc/index

Hello Mvc!

此时地址访问成功。

当项目访问成功后，那么问题来了，如果项目中存在静态资源文件，Handler 放行处理该如何配置，定义的拦截器怎么应用，此时关注 WebMvcConfigurationSupport 父类方法，重写相关方法即可。

```
// 静态资源 handler不进行处理 直接响应到客户端
@Override
public void configureDefaultServletHandling(DefaultServletHandlerConfigurer
configurer) {
    configurer.enable();
}

// 配置拦截器
@Bean
public LoginInterceptor loginInterceptor(){
    return new LoginInterceptor();
}

// 添加拦截器到mvc 环境
@Override
public void addInterceptors(InterceptorRegistry registry) {
    registry.addInterceptor(loginInterceptor());
}
```

5. Spring Boot 简介



5.1. 为什么使用 Spring Boot?

Spring 诞生时是 Java 企业版 (Java Enterprise Edition, JEE, 也称 J2EE) 的轻量级替代品。无需开发重量级的 Enterprise JavaBean (EJB), Spring 为企业级 Java 开发提供了一种相对简单的方法, 通过依赖注入和面向切面编程, 用简单的 Java 对象 (Plain Old Java Object, POJO) 实现了 EJB 的功能。

虽然 Spring 的组件代码是轻量级的, 但它的配置却是重量级的。

- 阶段一: XML配置

在 Spring 1.x 时代, 使用 Spring 开发都是 XML 配置的 Bean, 随着项目的扩大, 我们需要把 XML 配置文件放到不同的配置文件里, 那需要频繁的在开发的类和配置文件之间进行切换。

- 阶段二: 注解配置

在 Spring 2.x 时代，随着 JDK1.5 带来的注解支持，Spring 提供了声明 Bean 的注解（如 @Component、@Service 等），大大减少了配置量。

主要使用的方式是应用的基本配置（如数据库配置）用 XML，业务配置用注解。

- 阶段三：Java 配置

Spring 3.x 引入了基于 Java 配置类，这是一种类型安全的可重构配置方式，可以代替 XML。Spring 4.x 和 Spring Boot 都推荐使用 Java 配置。

所有这些配置都代表了开发时的损耗。因为在 Spring 特性配置和解决业务问题之间需要进行思维切换，所以写配置挤占了写应用程序逻辑的时间。除此之外，项目的依赖管理也是件很麻烦的事情，不仅要确定在项目中要用哪些库，而且你还要知道这些库的哪个版本和其他库不会有冲突。并且，依赖管理也是一种损耗，添加依赖不是写应用程序代码。一旦选错了依赖的版本，可能会产生版本不兼容的问题。

Spring-Boot 正是为了解决繁琐的代码配置而产生的，它让这些都成为了过去。

Spring Boot 简化了基于 Spring 的应用开发，只需要 "run" 就能创建一个独立的、生产级别的 Spring 应用。Spring Boot 为 Spring 平台及第三方库提供开箱即用的设置（提供默认设置），这样我们就可以使用 Spring Boot 快速启动一个项目。多数 Spring Boot 应用只需要很少的 Spring 配置。

5.2. 什么是 Spring Boot ?

随着 Ruby、Scala、NodeJs 等动态语言的流行，Java 开发显得相对笨重，配置繁琐、开发效率低下、部署流程复杂，而且集成第三方技术的难度也相对较大。

针对该环境，Spring Boot 被开发出来。Spring Boot 是一个在 Spring 的基础上搭建的全新的框架，它的设计目的是用来简化 Spring 应用的初始搭建以及开发过程。

Spring Boot 使用"约定大于配置"（项目中内置默认的配置，让你无须手动进行配置）的理念，减少了大量的配置文件的使用，可以让你的项目快速运行起来。借助 Spring Boot 在开发应用时，可以不用或很少去进行相关 XML 环境配置，简化了开发，大大提高项目开发效率。

5.3. Spring Boot 有什么好处？

简单、快速、方便。

思考一个问题：如果我们现在需要搭建一个 Spring MVC Web 项目，需要有哪些步骤呢？

1. 配置依赖
2. 配置 web.xml，加载 Spring 和 Spring MVC
3. 配置数据库连接
4. 配置 Spring 事务
5. 配置加载配置文件的读取，开启注解
6. 配置日志文件

...

7. 配置完成后部署Tomcat服务器

...

Spring Boot 特点

- 可以使用 SpringBoot 创建独立 Spring 应用程序
Spring Boot 可以以 jar 包的形式独立运行，运行 Spring Boot 项目只需通过 `java -jar xx.jar` 来运行。
- 嵌入式 Tomcat、Jetty 容器、无需部署 WAR 包
Spring Boot 可选择内嵌 Tomcat、Jetty 或者 Undertow，不需要依赖外部容器，这样我们无须以 war 包形式部署项目。
- 提供 starter 简化Maven
Spring Boot 中提供了一系列的 starter pom 来简化 Maven 的依赖加载。例如，当你使用了 `spring-boot-starter-web` 时，会自动加入 web 环境所需的依赖包。
- 自动配置 Spring 和第三方库
Spring Boot 会根据在类路径中的 jar 包、类，为 jar 包里的类自动配置 Bean，这样会极大地减少我们要使用的配置。当然，Spring Boot 只是考虑了大多数的开发场景，并不是所有的场景，若在实际开发中我们需要自动配置 Bean，而 Spring Boot 没有提供支持，则可以自定义自动配置。
- 提供生产就绪特征
Spring Boot 提供基于 http、ssh、telnet 对运行时的项目进行监控。如度量指标、健康检查及扩展配置等。
- 无代码生成和 XML 配置
Spring Boot 的神奇的不是借助于代码生成来实现的，而是通过条件注解来实现的，这是 Spring 4.x 提供的新特性。Spring 4.x 提倡使用 Java 配置和注解配置组合，而 Spring Boot 不需要任何 XML 配置即可实现 Spring 的所有配置。

6. Spring Boot 快速入门

6.1. 环境准备

IDEA、Maven、JDK1.8+、Spring Boot 2.x

6.2. 创建项目

通过 Maven 创建一个普通的 java 项目

6.3. 添加依赖坐标

```
<!-- SpringBoot的父级依赖。只有设置了parent标签，项目才是SpringBoot项目 -->
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.2.2.RELEASE</version>
</parent>

<dependencies>
```

```
<!-- web启动器，加载web环境所需要的依赖 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
</dependencies>
```

Spring Boot 的项目必须要设置 parent 标签，该 parent 包含了大量默认的配置，简化程序的开发。

spring-boot-starter-parent 是一个特殊的 starter，它用来提供相关的 Maven 默认依赖。使用它之后，可以省去常用包依赖的 version 标签。

6.4. 设置相关插件

```
<!-- SpringBoot项目的打包插件 -->
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

spring-boot-maven-plugin 插件是将 SpringBoot 应用程序打包成 Jar 包的插件。它可以将应用启动运行所需要的 Jar 包 都加载进来。当运行 "mvn package" 进行项目打包后，使用 "java -jar xxx.jar" 命令可以直接运行项目。

6.5. 添加源代码

```
@Controller
public class HelloController {
    @RequestMapping("hello")
    @ResponseBody
    public String hello(){
        return "Hello SpringBoot";
    }
}
```

6.6. 创建启动类

创建 StarterApplication.java 启动类

```

/**
 * SpringBoot 启动类
 */
@SpringBootApplication
public class Starter{
    public static void main(String[] args) {
        SpringApplication.run(Starter.class, args);
    }
}

```

SpringBoot 的启动类的作用是启动 SpringBoot 项目，是基于 Main 方法运行的。

注：启动类在启动时会扫描注解（@Controller、@Service、@Repository、@Component），扫描位置为同包及其子包下的注解，所以启动类的位置应放置于根包目录下。

@SpringBootApplication 组合注解

@SpringBootConfiguration：包含@Configuration，用于定义一个配置类。

@EnableAutoConfiguration：SpringBoot会自动根据jar包的依赖来自动配置项目。

@ComponentScan：告诉Spring哪个packages的注解标识的类会被Spring自动扫描并且装入bean容器。

6.7. 启动应用

运行 main 方法

```

main] com.shsxt.StarterApplication : Starting StarterApplication on 3FDE with PID 43100 (C:\IdeaWorkspace\
main] com.shsxt.StarterApplication : No active profile set, falling back to default profiles: default
main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.29]
main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
main] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization completed in 935 ms
main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
main] com.shsxt.StarterApplication : Started StarterApplication in 1.593 seconds (JVM running for 2.548)

```

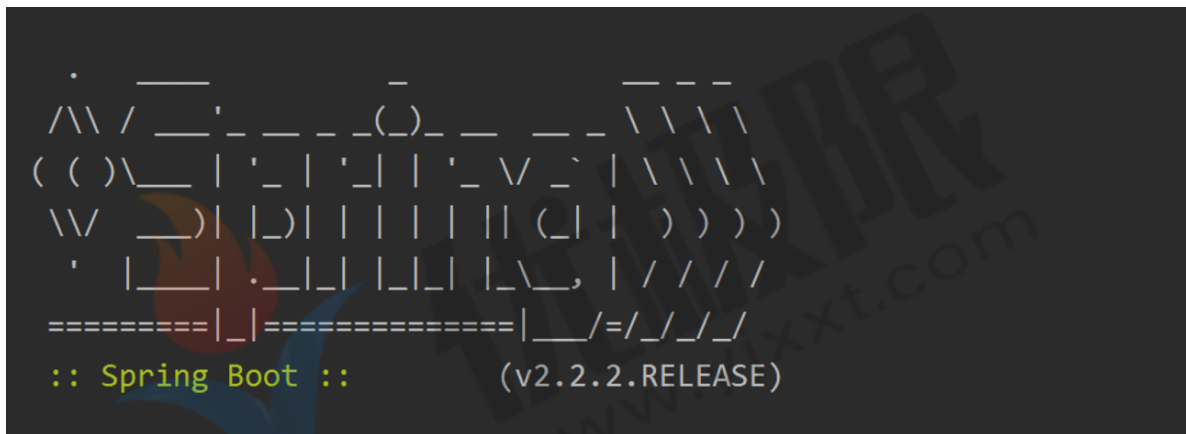
通过浏览器访问 <http://localhost:8080/hello> 效果如下:

← → ↺ ⓘ localhost:8080/hello

Hello SpringBoot

7. Spring Boot 核心配置

7.1. 设置 Banner 图标



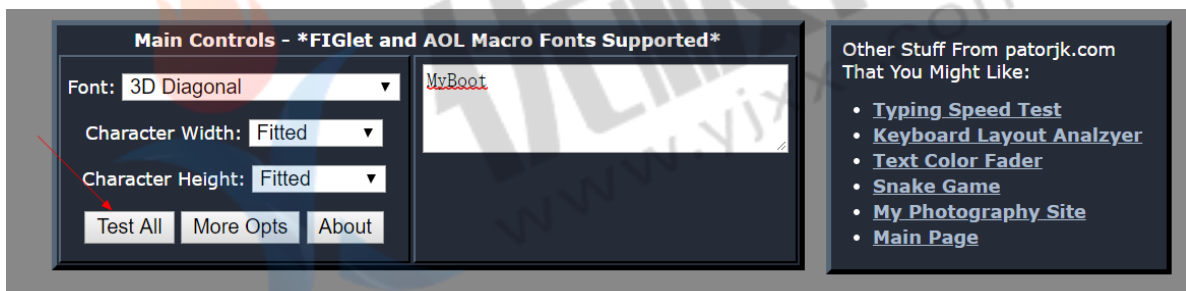
在搭建 Spring Boot 项目环境时，程序启动后会在控制台打印醒目的 SpringBoot 图标，图标描述了 Spring Boot 版本信息，这是 Spring Boot 项目与 Spring 项目启动区别较大的地方，Spring Boot 通过默认 Banner 在程序启动时显示应用启动图标，当然图标我们也可以进行自定义。

7.1.1. Banner 图标自定义

Spring Boot 项目启动时默认加载 src/main/resources 目录下的 banner.txt 图标文件，如果该目录文件未提供，则使用 Spring Boot 默认。在 main 目录下新建 resources 资源目录，并在该目录下新建 banner.txt 文本文件，可以设置自定义图标。

打开网址: <http://patorjk.com/software/taag/#p=display&f=Graffiti&t=Type%20Something>

在线生成图标对应文本并将文本内容copy 到 banner.txt 中



启动Spring Boot 应用打印如下:



7.1.2. Banner 图标关闭

如果启动时不想要看到启动图标，这里也可以通过代码进行关闭操作，修改 StarterApplication 设置 BannerMode 值为 Banner.Mode.OFF，启动 Spring Boot 应用关闭图标输出功能即可

```
@SpringBootApplication
public class StarterApplication {

    public static void main(String[] args) {
        SpringApplication springApplication = new
        SpringApplication(StarterApplication.class);
        // 设置 banner 图标关闭
        springApplication.setBannerMode(Banner.Mode.OFF);
        springApplication.run();
    }
}
```

7.2. Spring Boot 配置文件

Spring Boot 默认会读取全局配置文件，配置文件名固定为：application.properties 或 application.yml，放置在 src/main/resources 资源目录下，使用配置文件来修改 SpringBoot 自动配置的默认值。

在 resources 资源目录下添加 application.properties 文件，配置信息如下：

```
## 项目启动端口号配置
server.port=8989
## 项目访问上下文路径
server.servlet.context-path=/mvc

## 数据源配置
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://127.0.0.1:3306/hr?
useUnicode=true&characterEncoding=utf8
spring.datasource.username=root
spring.datasource.password=root
```

或者 application.yml 文件

```
## 端口号 上下文路径
server:
  port: 8989
  servlet:
    context-path: /mvc

## 数据源配置
spring:
  datasource:
    type: com.mchange.v2.c3p0.ComboPooledDataSource
    driver-class-name: com.mysql.cj.jdbc.Driver
    url: jdbc:mysql://127.0.0.1:3306/hr
    username: root
    password: root
```

7.3. Starter 坐标 & 自动化配置

7.3.1. Starter坐标配置

Spring Boot 引入了全新的Starter坐标体系，简化企业项目开发大部分场景的 Starter pom，应用程序引入指定场景的 Start pom 相关配置就可以消除，通过 Spring Boot 就可以得到自动配置的 Bean。

7.3.1.1. Web starter

使用 Spring MVC 来构建 RESTful Web 应用，并使用 Tomcat 作为默认内嵌容器

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

7.3.1.2. Freemarker Starter & Thymeleaf starter

集成视图技术，引入 Freemarker Starter , Thymeleaf Starter

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-freemarker</artifactId>
</dependency>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

7.3.1.3. JavaMail邮件发送 Starter

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

7.3.1.4. 引入AOP环境

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

相关starter系列坐标参考

名称	描述
spring-boot-starter	核心Spring Boot starter，包括自动配置支持，日志和YAML
spring-boot-starter-actuator	生产准备的特性，用于帮我们监控和管理应用
spring-boot-starter-amqp	对“高级消息队列协议”的支持，通过spring-rabbit实现
spring-boot-starter-aop	对面向切面编程的支持，包括spring-aop和AspectJ
spring-boot-starter-batch	对Spring Batch的支持，包括HSQLDB数据库
spring-boot-starter-cloud-connectors	对Spring Cloud Connectors的支持，简化在云平台下（例如，Cloud Foundry 和Heroku）服务的连接
spring-boot-starter-data-elasticsearch	对Elasticsearch搜索和分析引擎的支持，包括spring-data-elasticsearch
spring-boot-starter-data-gemfire	对GemFire分布式数据存储的支持，包括spring-data-gemfire
spring-boot-starter-data-jpa	对“Java持久化API”的支持，包括spring-data-jpa，spring-orm和Hibernate
spring-boot-starter-data-mongodb	对MongoDB NOSQL数据库的支持，包括spring-data-mongodb
spring-boot-starter-data-rest	对通过REST暴露Spring Data仓库的支持，通过spring-data-rest-webmvc实现
spring-boot-starter-data-solr	对Apache Solr搜索平台的支持，包括spring-data-solr
spring-boot-starter-freemarker	对FreeMarker模板引擎的支持
spring-boot-starter-groovy-templates	对Groovy模板引擎的支持
spring-boot-starter-hateoas	对基于HATEOAS的RESTful服务的支持，通过spring-hateoas实现
spring-boot-starter-hornetq	对“Java消息服务API”的支持，通过HornetQ实现
spring-boot-starter-integration	对普通spring-integration模块的支持
spring-boot-starter-jdbc	对JDBC数据库的支持
spring-boot-starter-jersey	对Jersey RESTful Web服务框架的支持
spring-boot-starter-jta-atomikos	对JTA分布式事务的支持，通过Atomikos实现

名称	描述
spring-boot-starter-jta-bitronix	对JTA分布式事务的支持，通过Bitronix实现
spring-boot-starter-mail	对javax.mail的支持
spring-boot-starter-mobile	对spring-mobile的支持
spring-boot-starter-mustache	对Mustache模板引擎的支持
spring-boot-starter-redis	对REDIS键值数据存储的支持，包括spring-redis
spring-boot-starter-security	对spring-security的支持
spring-boot-starter-social-facebook	对spring-social-facebook的支持
spring-boot-starter-social-linkedin	对spring-social-linkedin的支持
spring-boot-starter-social-twitter	对spring-social-twitter的支持
spring-boot-starter-test	对常用测试依赖的支持，包括JUnit, Hamcrest和Mockito，还有spring-test模块
spring-boot-starter-thymeleaf	对Thymeleaf模板引擎的支持，包括和Spring的集成
spring-boot-starter-velocity	对Velocity模板引擎的支持
spring-boot-starter-web	对全栈web开发的支持，包括Tomcat和spring-webmvc
spring-boot-starter-websocket	对WebSocket开发的支持
spring-boot-starter-ws	对Spring Web服务的支持

传统的 `maven` 坐标这里同样适用，如果引入传统 `maven` 坐标需要考虑相关配置类的编写。

如果引入相关starter坐标这里不存在，可以到[这里](#)搜索。

7.3.2. 自动化配置

7.3.2.1. SpringBoot Starter坐标版本查看

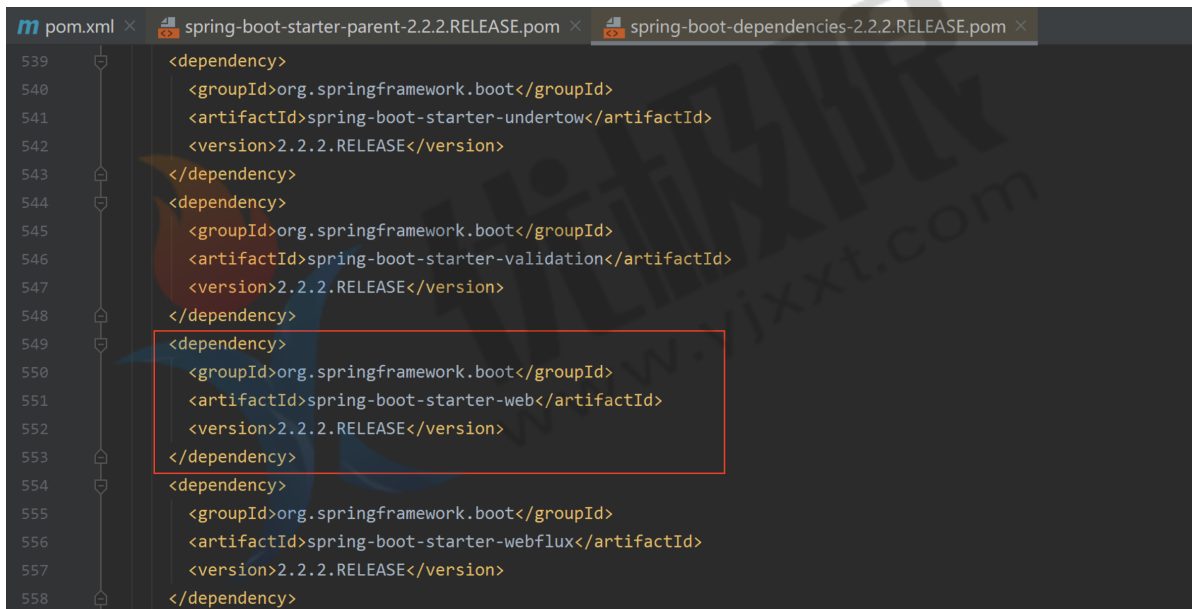
前面介绍了 SpringBoot Starter 相关坐标，引入 Starter 坐标来简化应用环境的配置。这里以环境搭建 `spring-boot-starter-web` 坐标来简单分析 SpringBoot 自动化配置过程。

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

这里引入的 web 环境坐标不像传统的maven坐标那样包含坐标的版本号，项目中引入的 starter 系列坐标对应的版本库统一由父工程坐标统一控制即项目中引入的 parent 标签。

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <!--
    父类项目统一对项目依赖版本统一控制！
  -->
  <version>2.2.2.RELEASE</version>
</parent>
```

这里 spring-boot-starter-parent 继承 spring-boot-dependencies 项目，在 spring-boot-dependencies 项目中定义了 spring-boot-starter-web 坐标的版本！（spring-boot-dependencies 项目中定义了当前 SpringBoot版本下各个 starter 坐标版本以及依赖的其他坐标版本）



```
m pom.xml x spring-boot-starter-parent-2.2.2.RELEASE.pom x spring-boot-dependencies-2.2.2.RELEASE.pom x
539      <dependency>
540        <groupId>org.springframework.boot</groupId>
541        <artifactId>spring-boot-starter-undertow</artifactId>
542        <version>2.2.2.RELEASE</version>
543      </dependency>
544      <dependency>
545        <groupId>org.springframework.boot</groupId>
546        <artifactId>spring-boot-starter-validation</artifactId>
547        <version>2.2.2.RELEASE</version>
548      </dependency>
549      <dependency>
550        <groupId>org.springframework.boot</groupId>
551        <artifactId>spring-boot-starter-web</artifactId>
552        <version>2.2.2.RELEASE</version>
553      </dependency>
554      <dependency>
555        <groupId>org.springframework.boot</groupId>
556        <artifactId>spring-boot-starter-webflux</artifactId>
557        <version>2.2.2.RELEASE</version>
558      </dependency>
```

```
m pom.xml x spring-boot-starter-parent-2.2.2.RELEASE.pom x spring-boot-dependencies-2.2.2.RELEASE.pom x
27 <properties>
28 <activemq.version>5.15.11</activemq.version>
29 <antlr2.version>2.7.7</antlr2.version>
30 <appengine-sdk.version>1.9.77</appengine-sdk.version>
31 <artemis.version>2.10.1</artemis.version>
32 <aspectj.version>1.9.5</aspectj.version>
33 <assertj.version>3.13.2</assertj.version>
34 <atomikos.version>4.0.6</atomikos.version>
35 <awaitility.version>4.0.1</awaitility.version>
36 <bitronix.version>2.1.4</bitronix.version>
37 <build-helper-maven-plugin.version>3.0.0</build-helper-maven-plugin.version>
38 <byte-buddy.version>1.10.4</byte-buddy.version>
39 <caffeine.version>2.8.0</caffeine.version>
40 <cassandra-driver.version>3.7.2</cassandra-driver.version>
41 <classmate.version>1.5.1</classmate.version>
42 <commons-codec.version>1.13</commons-codec.version>
43 <commons-dbcp2.version>2.7.0</commons-dbcp2.version>
44 <commons-lang3.version>3.9</commons-lang3.version>
45 <commons-pool.version>1.6</commons-pool.version>
46 <commons-pool2.version>2.7.0</commons-pool2.version>
47 <couchbase-cache-client.version>2.1.0</couchbase-cache-client.version>
48 <couchbase-client.version>2.7.11</couchbase-client.version>
49 <db2-jdbc.version>11.5.0.0</db2-jdbc.version>
50 <dependency-management-plugin.version>1.0.8.RELEASE</dependency-management-plugin.version>
```

这里依赖的坐标较多，不在截图展示，可关注 spring-boot-dependencies-2.2.2.RELEASE.pom 文件。

7.3.2.2. Spring Boot自动化配置

Spring Boot的项目一般都会有 *Application 的入口类，入口类中提供 main 方法，这是一个标准的 Java 应用程序的入口方法。@SpringBootApplication 注解是 Spring Boot 的核心注解，它其实是一个组合注解：

- @SpringBootApplication

```
Starter.java x SpringBootApplication.java x
48 @Target(ElementType.TYPE)
49 @Retention(RetentionPolicy.RUNTIME)
50 @Documented
51 @Inherited
52 @SpringBootConfiguration
53 @EnableAutoConfiguration
54 @ComponentScan(excludeFilters = { @Filter(type = FilterType.CUSTOM, classes = TypeExcludeFilter.class),
55     @Filter(type = FilterType.CUSTOM, classes = AutoConfigurationExcludeFilter.class) })
56 public @interface SpringBootApplication {
57
```

可以看出该注解也是一个组合注解，组合了 @Configuration 注解，对于Spring Boot应用，@SpringBootConfiguration 注解属于Boot 项目的配置注解也是属于一个组合注解，Spring Boot 项目中推荐使用@SpringBootConfiguration 注解，因为其组合了 @Configuration 注解。

- @EnableAutoConfiguration

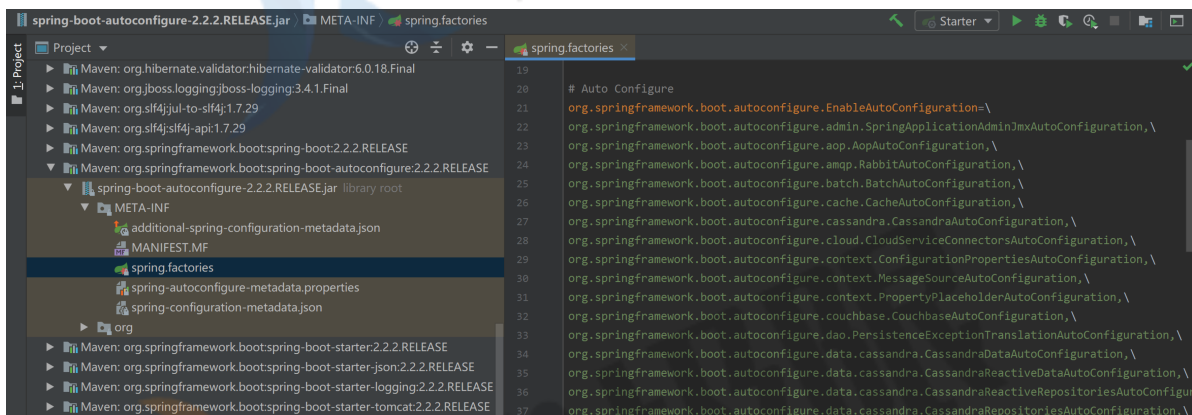
```
Starter.java x SpringBootApplication.java x EnableAutoConfiguration.java x
78 @Target(ElementType.TYPE)
79 @Retention(RetentionPolicy.RUNTIME)
80 @Documented
81 @Inherited
82 @AutoConfigurationPackage
83 @Import(AutoConfigurationImportSelector.class)
84 public @interface EnableAutoConfiguration {
85
```

@EnableAutoConfiguration 注解组合了 @AutoConfigurationPackage、@Import(AutoConfigurationImportSelector.class) 注解。

@AutoConfigurationPackage 底层也是一个
@Import(AutoConfigurationPackages.Registrar.class), 其会把启动类的包下组件都扫描到Spring容器中。

```
36 @Target(ElementType.TYPE)
37 @Retention(RetentionPolicy.RUNTIME)
38 @Documented
39 @Inherited
40 @Import(AutoConfigurationPackages.Registrar.class)
41 public @interface AutoConfigurationPackage {
42
```

@Import(AutoConfigurationImportSelector.class) 自动配置的核心类
AutoConfigurationImportSelector.class, 该类导入大量的自动配置类, debug可以发现, 其读取的是
classpath下的 META-INF/spring.factories 下配置文件。



以 WebMvcAutoConfiguration 为例, 可以看出该类使用 @Configuration 注解进行标注其为一个配置类。

```
135 @Configuration(proxyBeanMethods = false)
136 @ConditionalOnWebApplication(type = Type.SERVLET)
137 @ConditionalOnClass({ Servlet.class, DispatcherServlet.class, WebMvcConfigurer.class })
138 @ConditionalOnMissingBean(WebMvcConfigurationSupport.class)
139 @AutoConfigureOrder(Ordered.HIGHEST_PRECEDENCE + 10)
140 @AutoConfigureAfter({ DispatcherServletAutoConfiguration.class, TaskExecutionAutoConfiguration.class,
141     ValidationAutoConfiguration.class })
142 public class WebMvcAutoConfiguration {
143
144     public static final String DEFAULT_PREFIX = "";
145
146     public static final String DEFAULT_SUFFIX = "";
147
148     private static final String[] SERVLET_LOCATIONS = { "/" };
```

当然 spring.factories 文件中配置类默认不会都生效, 具体哪些配置类生效由配置类上标注的
@ConditionalOnClass 注解来决定, 这里了解下 @ConditionalOnClass 注解含义。

@ConditionalOnClass: 有条件的情况下会加载class内容, 返回boolean类型, 检测当前项目环境中是否具备了某个技术环境, 如果存在的情况下会提供当前环境的自动化配置, 不存在则不

@ConditionalOnBean	// 当给定的在bean存在时, 则实例化当前Bean
@ConditionalOnMissingBean	// 当给定的在bean不存在时, 则实例化当前Bean
@ConditionalOnClass	// 当给定的类名在类路径上存在, 则实例化当前Bean
@ConditionalOnMissingClass	// 当给定的类名在类路径上不存在, 则实例化当前Bean

意味着 WebMvcAutoConfiguration 配置类生效需要环境中存在 Servlet.class,
DispatcherServlet.class, WebMvcConfigurer.class 实例, 配置类才会生效。

从以上分析可以得出如下结论:

Spring Boot 通过 maven 中的 starter 导入了所需场景下的 jar 包, 并通过主启动类上的 @SpringBootApplication 中的 @EnableAutoConfiguration 读取了类路径下的 META-INF/spring.factories 下 EnableAutoConfiguration 的配置类, 这些配置类使用 @ConditionalOnClass 来标注, 根据 @ConditionalOnClass 标注的约束条件来引入自动化的环境配置。

7.4. Profile 配置

Profile 是 Spring 用来针对不同环境对不同配置提供支持的全局 Profile 配置使用 application-{profile}.yml, 比如 application-dev.yml, application-test.yml。

通过在 application.yml 中设置 spring.profiles.active=test|dev|prod 来动态切换不同环境, 具体配置如下:

- application-dev.yml 开发环境配置文件

```
server:
  port: 8989
```

- application-test.yml 测试环境配置文件

```
server:
  port: 9999
```

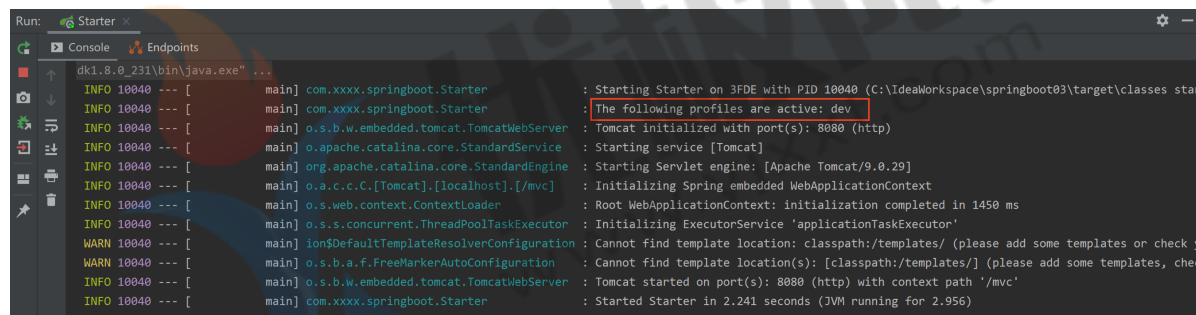
- application-prod.yml 生产环境配置文件

```
server:
  port: 8686
```

- application.yml 主配置文件

```
## 环境选择配置
spring:
  profiles:
    active: dev
```

启动Starter 查看控制台输入效果

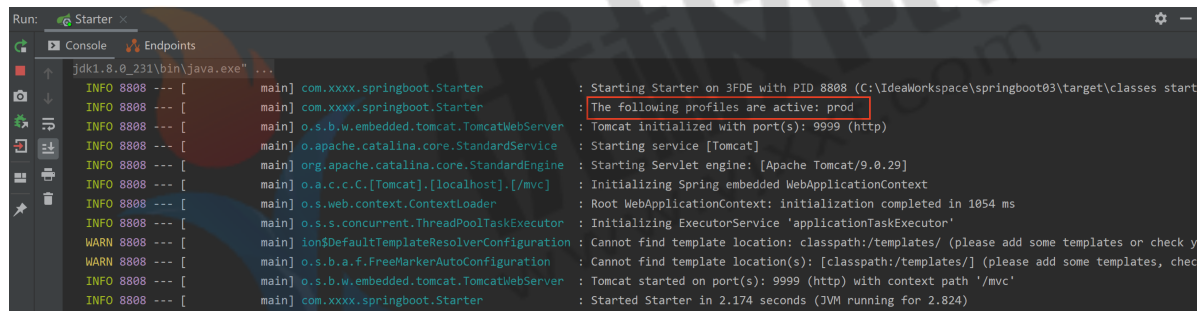


```
Run: Starter
Console
Endpoints
dki1.8.0_231\bin\java.exe" ...
INFO 10040 --- [main] com.xxxx.springboot.Starter : Starting Starter on 3FDE with PID 10040 (C:\IdeaWorkspace\springboot03\target\classes
INFO 10040 --- [main] com.xxxx.springboot.Starter : The following profiles are active: dev
INFO 10040 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
INFO 10040 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
INFO 10040 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.29]
INFO 10040 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/mvc] : Initializing Spring embedded WebApplicationContext
INFO 10040 --- [main] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization completed in 1450 ms
INFO 10040 --- [main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
WARN 10040 --- [main] ion$DefaultTemplateResolverConfiguration : Cannot find template location: classpath:/templates/ (please add some templates or check y
WARN 10040 --- [main] o.s.b.a.f.FreeMarkerAutoConfiguration : Cannot find template location(s): [classpath:/templates/] (please add some templates, che
INFO 10040 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path '/mvc'
INFO 10040 --- [main] com.xxxx.springboot.Starter : Started Starter in 2.241 seconds (JVM running for 2.956)
```

修改 application.yml 设置 active 值为 prod

```
## 环境选择配置
spring:
  profiles:
    active: prod
```

启动Starter 再次查看控制台输入效果



```
Run: Starter
jkd1.8.0_231\bin\java.exe ...
INFO 8808 --- [main] com.xxx.springboot.Starter : Starting Starter on 3FDE with PID 8808 (C:\IdeaWorkspace\springboot03\target\classes start
INFO 8808 --- [main] com.xxx.springboot.Starter : The following profiles are active: prod
INFO 8808 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 9999 (http)
INFO 8808 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
INFO 8808 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.29]
INFO 8808 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/mvc] : Initializing Spring embedded WebApplicationContext
INFO 8808 --- [main] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization completed in 1054 ms
INFO 8808 --- [main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
WARN 8808 --- [main] ion$DefaultTemplateResolverConfiguration : Cannot find template location: classpath:/templates/ (please add some templates or check y
WARN 8808 --- [main] o.s.b.a.f.FreeMarkerAutoConfiguration : Cannot find template location(s): [classpath:/templates/] (please add some templates, che
INFO 8808 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 9999 (http) with context path '/mvc'
INFO 8808 --- [main] com.xxx.springboot.Starter : Started Starter in 2.174 seconds (JVM running for 2.824)
```

7.5. 日志配置

在开发企业项目时，日志的输出对于系统 bug 定位无疑是一种比较有效的方式，也是项目后续进入生产环境后快速发现错误解决错误的一种有效手段，所以日志的使用对于项目也是比较重要的一块功能。

Spring Boot 默认使用 LogBack 日志系统，如果不需要更改为其他日志系统如 Log4j2 等，则无需多余的配置，LogBack 默认将日志打印到控制台上。如果要使用 LogBack，原则上是需要添加 dependency 依赖的。

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-logging</artifactId>
</dependency>
```

```
► Maven: org.attoparser:attoparser:2.0.5.RELEASE
► Maven: org.freemarker:freemarker:2.3.29
► Maven: org.hibernate.validator:hibernate-validator:6.0.18.Final
► Maven: org.jboss.logging:jboss-logging:3.4.1.Final
► Maven: org.slf4j:slf4j-api:1.7.29
► Maven: org.slf4j:slf4j-api:1.7.29
► Maven: org.springframework.boot:spring-boot:2.2.2.RELEASE
```

因为新建的 Spring Boot 项目一般都会引用 spring-boot-starter 或者 spring-boot-starter-web，而这两个起步依赖中都已经包含了对于 spring-boot-starter-logging 的依赖，所以，无需额外添加依赖。

7.5.1. 项目中日志信息输出

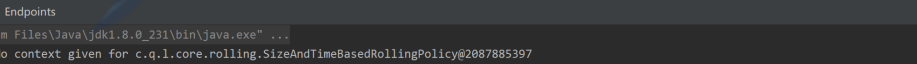
Starter 启动类中添加 Log 日志类，控制台打印日志信息。

```
package com.xxx;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

[illegible]

```
logging:
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger- %msg%n"
    level: debug
  file:
    path: "."
    name: "springboot.log"
```



```

Run: Starter x
Console Endpoints
"C:\Program Files\Java\jdk1.8.0_231\bin\java.exe" ...
LOGBACK: No context given for c.q.l.core.rolling.SizeAndTimeBasedRollingPolicy@2687885397
2020-05-12 15:43:38 [main] INFO com.xxx.springboot.Starter- Starting Starter on 3FDE with PID 7120 (C:\IdeaWorkspace\springboot03\target\classes started by ls in C:\IdeaWorkspace\springboot03\target\classes)
2020-05-12 15:43:38 [main] INFO com.xxx.springboot.Starter- The following profiles are active: test
2020-05-12 15:43:39 [main] INFO org.springframework.boot.web.embedded.tomcat.TomcatWebServer- Tomcat initialized with port(s): 8989 (http)
2020-05-12 15:43:39 [main] INFO org.apache.catalina.core.StandardService- Starting service [Tomcat]
2020-05-12 15:43:39 [main] INFO org.apache.catalina.core.StandardEngine- Starting Servlet engine: [Apache Tomcat/9.0.29]
2020-05-12 15:43:39 [main] INFO org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/mvc]- Initializing Spring embedded WebApplicationContext
2020-05-12 15:43:39 [main] INFO org.springframework.web.context.ContextLoader- Root WebApplicationContext: initialization completed in 1148 ms
2020-05-12 15:43:39 [main] INFO org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor- Initializing ExecutorService 'applicationTaskExecutor'
2020-05-12 15:43:39 [main] WARN org.springframework.boot.autoconfigure.thymeleaf.ThymeleafAutoConfiguration.DefaultTemplateResolverConfiguration- Cannot find template location: classpath:/templates/
2020-05-12 15:43:39 [main] WARN org.springframework.boot.autoconfigure.freemarker.FreeMarkerAutoConfiguration- Cannot find template location(s): [classpath:/templates/
2020-05-12 15:43:40 [main] INFO org.springframework.boot.web.embedded.tomcat.TomcatWebServer- Tomcat started on port(s): 8989 (http) with context path '/mvc'
2020-05-12 15:43:40 [main] INFO org.springframework.boot.SpringApplication- Started application in 1.5 seconds (JVM running for 1.5 seconds)

```

8. Freemarker & Thymeleaf 视图集成

8.1. Freemarker 视图集成

SpringBoot 内部支持 Freemarker 视图技术的集成，并提供了自动化配置类 FreeMarkerAutoConfiguration，借助自动化配置可以很方便的集成 Freemarker 基础到 SpringBoot 环境中。这里借助入门项目引入 Freemarker 环境配置。

- Starter 坐标引入

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-freemarker</artifactId>
</dependency>
```

- 添加 Freemarker 配置信息

Freemarker 默认视图路径 resources/templates 目录(由自动化配置类 FreemarkerProperties 决定)，该目录可以进行在 application.yml 中进行修改。

```
32 @ConfigurationProperties(prefix = "spring.freemarker")
33 public class FreemarkerProperties extends AbstractTemplateViewResolverProperties {
34
35     public static final String DEFAULT_TEMPLATE_LOADER_PATH = "classpath:/templates/";
36
37     public static final String DEFAULT_PREFIX = "";
38
39     public static final String DEFAULT_SUFFIX = ".ftlh";
40
```

修改 application.yml 添加 freemarker 基本配置如下：

```
spring:
  freemarker:
    suffix: .ftl
    content-type: text/html #响应格式
    charset: UTF-8
    template-loader-path: classpath:/views/
```

- 编写 IndexController 控制器转发视图

```
package com.xxxx.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

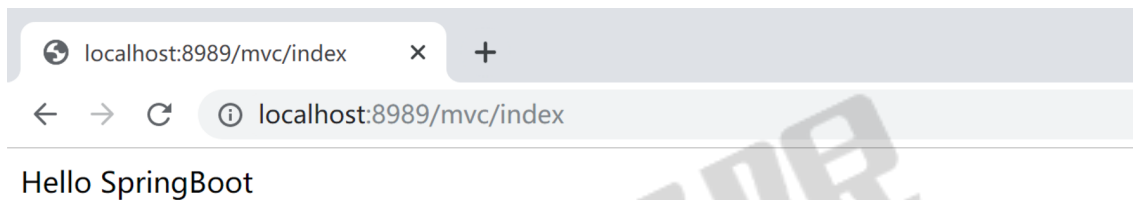
@Controller
public class IndexController {

    @RequestMapping("index")
    public String index(){
        return "index";
    }
}
```

- views 目录下添加 index.ftl 视图

```
index.ftl
1 Hello SpringBoot
```

- 启动 Starter 访问



8.2. Thymeleaf 视图集成

SpringBoot 支持多种视图技术集成, 并且 SpringBoot 官网推荐使用 Thymeleaf 作为前端视图页面, 这里实现 Thymeleaf 视图集成, 借助入门项目引入 Thymeleaf 环境配置。

- starter坐标引入

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

- 添加 Thymeleaf 配置信息

Thymeleaf 默认视图路径 resources/templates 目录(由自动化配置类 ThymeleafProperties 类决定), 该目录可以进行在 application.yml 中进行修改。

```
37     @ConfigurationProperties(prefix = "spring.thymeleaf")
38     public class ThymeleafProperties {
39
40         private static final Charset DEFAULT_ENCODING = StandardCharsets.UTF_8;
41
42         public static final String DEFAULT_PREFIX = "classpath:/templates/";
43
44         public static final String DEFAULT_SUFFIX = ".html";
45     }
```

```
## 环境选择配置
spring:
  thymeleaf:
    prefix: classpath:/html/
    ## 关闭 thymeleaf 页面缓存
    cache: false
```

- 编写 IndexController 控制器转发视图

```
@Controller
public class IndexController {

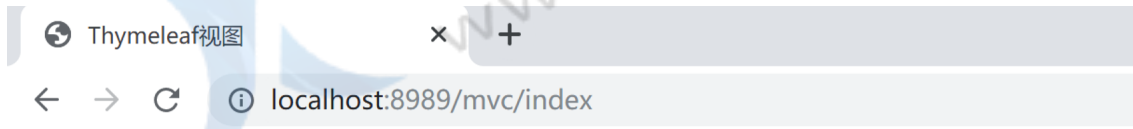
    @RequestMapping("index")
    public String index(Model model){
        // 设置请求域的值
        model.addAttribute("msg", "Hello SpringBoot");
        return "index";
    }
}
```

- html 目录下添加 index.html 视图

修改 Thymeleaf 模板默认存放路径 (在 resources 目录下创建 html 文件夹)

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Thymeleaf视图</title>
</head>
<body>
  <!-- 获取请求域的值 -->
  <h2 th:text="${msg}"></h2>
</body>
</html>
```

- 启动Starter访问



Hello SpringBoot

9. SpringBoot 静态资源访问

从入门项目中可以看到：对于 Spring Mvc 请求拦截规则为 '/', Spring Boot 默认静态资源路径如下：

```
38 @ConfigurationProperties(prefix = "spring.resources", ignoreUnknownFields = false)
39 public class ResourceProperties {
40
41     private static final String[] CLASSPATH_RESOURCE_LOCATIONS = { "classpath:/META-INF/resources/",
42         "classpath:/resources/", "classpath:/static/", "classpath:/public/" };
43
44     /**
45      * Locations of static resources. Defaults to classpath:[/META-INF/resources/,
46      * /resources/, /static/, /public/].
47      */
48     private String[] staticLocations = CLASSPATH_RESOURCE_LOCATIONS;
49
50     /**
51      * Whether to enable default resource handling.
52      */
53     private boolean addMappings = true;
54
55     private final Chain chain = new Chain();
56
57     private final Cache cache = new Cache();
```

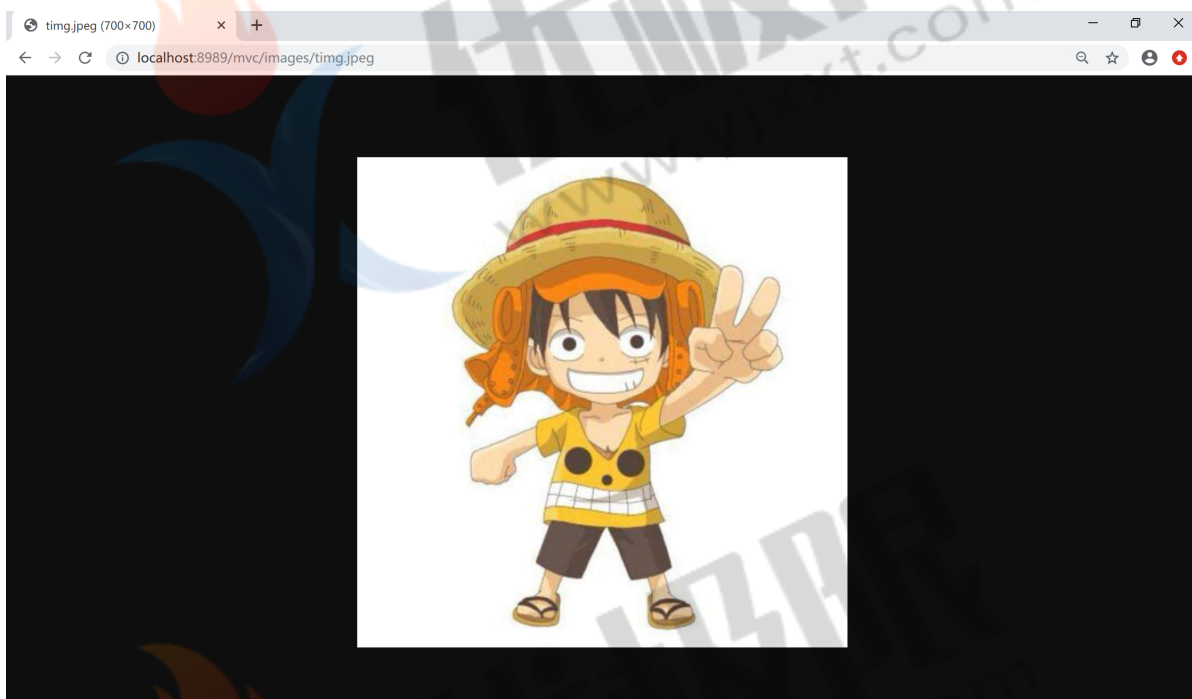
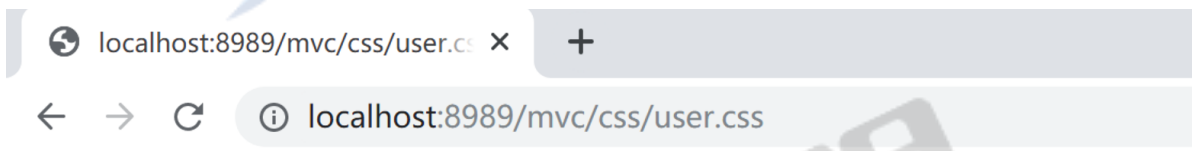
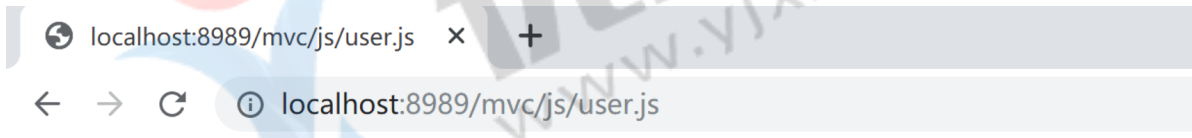
即：我们可以在 resources 资源目录下存放 web 应用静态资源文件。

9.1. 默认静态资源路径

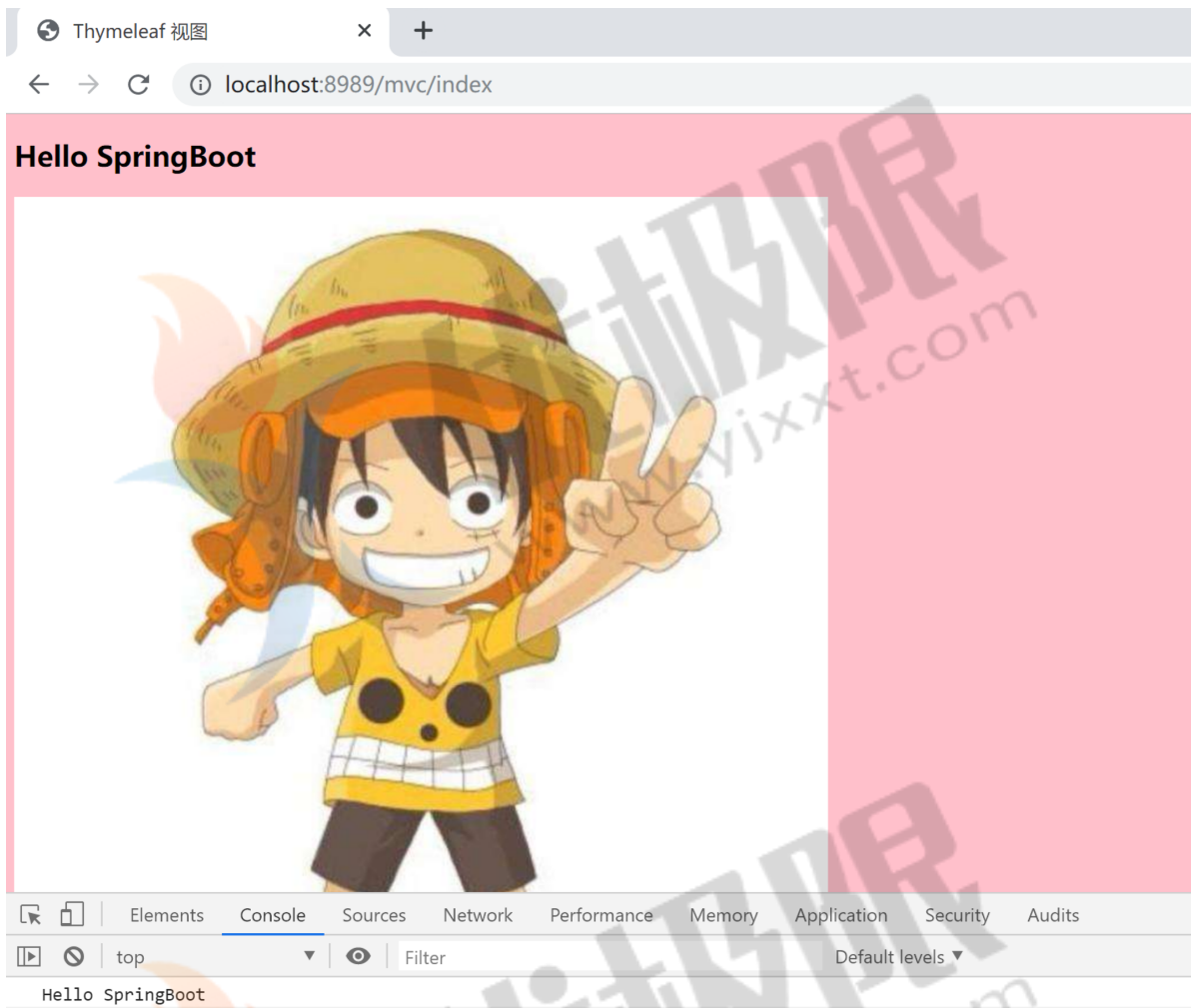
在 resources 目录下创建 static 或者 public 目录，存放 images、js、css 等静态资源文件



浏览器访问:



在视图中使用静态资源



9.2. 自定义静态资源路径

在 `spring.resources.static-locations` 后面追加一个配置 `classpath:/os/`

```
spring:
  # 修改默认的静态寻址资源目录 多个路径之间用逗号隔开
  resources:
    static-locations: classpath:/public/,classpath:/static/,classpath:/os/
```

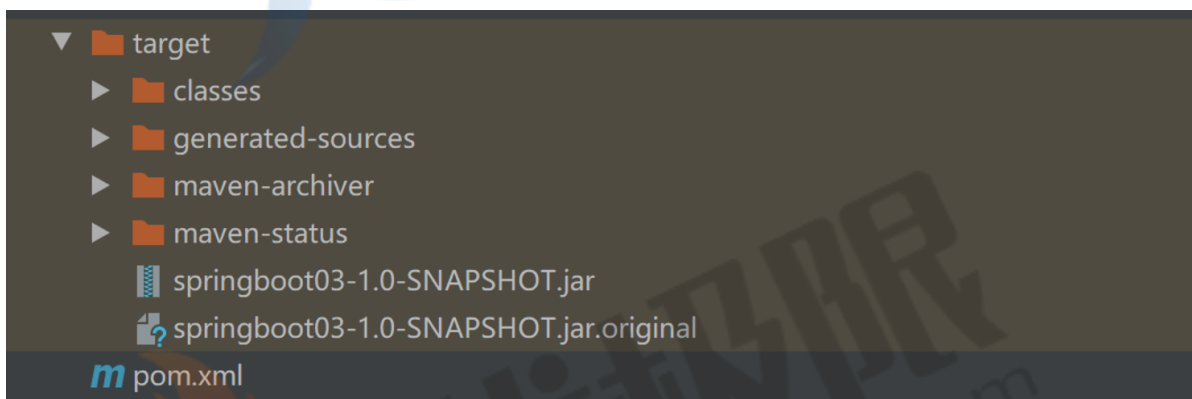
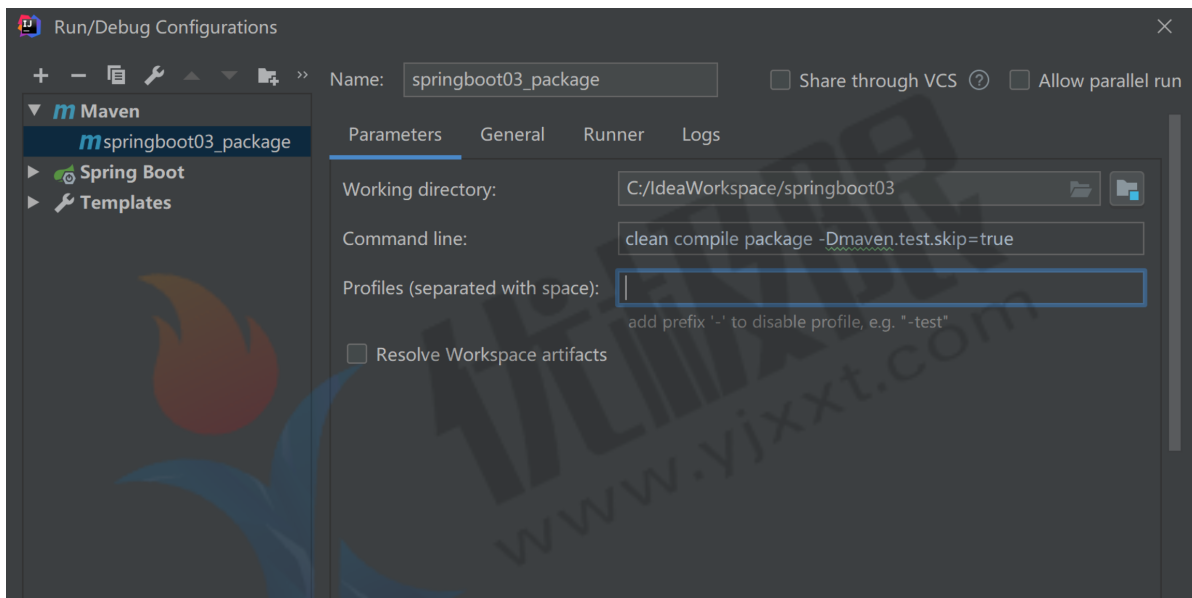
10. SpringBoot 应用打包与部署

当项目开发完毕进行部署上线时，需要对项目进行打包操作，入门中构建的项目属于普通应用，由于 SpringBoot 内嵌 Tomcat 容器，所有打包后的 jar 包默认可以自行运行。

10.1. Jar 包部署

10.1.1. 配置打包命令

idea 下配置 `clean compile package -Dmaven.test.skip=true` 执行打包命令，target 目录得到待部署的项目文件。



10.1.2. 部署并访问

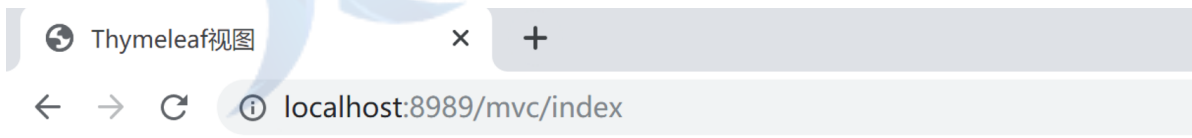
打开本地 dos 窗口，执行 `java -jar` 命令 部署已打好的 jar 包文件。

命令如下： `java -jar jar包所在目录`



```
C:\Windows\system32\cmd.exe - java -jar C:\IdeaWorkspace\springboot03\target\springboot03-1.0-SNAPSHOT.jar
17:04:35.303 [main] INFO com.xxxx.springboot.Starter - SpringBoot 应用正在启动...
LOGBACK: No context given for c.q.l.core.rolling.SizeAndTimeBasedRollingPolicy@754666084
17:04:36 [main] INFO com.xxxx.springboot.Starter- Starting Starter v1.0-SNAPSHOT on 3FDE with PID 9916 (C:\IdeaWork
space\springboot03\target\springboot03-1.0-SNAPSHOT.jar started by ls in C:\Users\ls)
17:04:36 [main] INFO com.xxxx.springboot.Starter- The following profiles are active: test
17:04:38 [main] INFO org.springframework.boot.web.embedded.tomcat.TomcatWebServer- Tomcat initialized with port(s):
8989 (http)
17:04:38 [main] INFO org.apache.catalina.core.StandardService- Starting service [Tomcat]
17:04:38 [main] INFO org.apache.catalina.core.StandardEngine- Starting Servlet engine: [Apache Tomcat/9.0.29]
17:04:38 [main] INFO org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/mvc]- Initializing Spring embedd
ed WebApplicationContext
17:04:38 [main] INFO org.springframework.web.context.ContextLoader- Root WebApplicationContext: initialization comp
leted in 1906 ms
17:04:39 [main] INFO org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor- Initializing ExecutorService
'applicationTaskExecutor'
17:04:39 [main] INFO org.springframework.boot.autoconfigure.web.servlet>WelcomePageHandlerMapping- Adding welcome p
age template: index
17:04:39 [main] INFO org.springframework.boot.web.embedded.tomcat.TomcatWebServer- Tomcat started on port(s): 8989
(http) with context path '/mvc'
17:04:39 [main] INFO com.xxxx.springboot.Starter- Started Starter in 3.857 seconds (JVM running for 4.485)
```

浏览器访问



Hello SpringBoot

10.2. war 包部署

War 包形式部署 Web 项目在生产环境中是比较常见的部署方式，也是目前大多数 web 应用部署的方案，这里对于 Spring Boot Web 项目进行打包部署步骤如下

10.2.1. pom.xml修改

- 应用类型修改

由于入门项目构建的项目默认为 jar 应用，所以这里打 war 需要作如下修改

```
m pom.xml x
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5   <modelVersion>4.0.0</modelVersion>
6
7   <groupId>com.xxxx</groupId>
8   <artifactId>springboot03</artifactId>
9   <version>1.0-SNAPSHOT</version>
10  <packaging>war</packaging>
11
```

- 内嵌tomcat忽略

构建 SpringBoot 应用时，引入的 spring-boot-starter-web 默认引入 Tomcat 容器，这里忽略掉内容 Tomcat

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-tomcat</artifactId>
  <scope>provided</scope>
</dependency>
```

```

<build>
  <!--
    配置生成的war文件名
  -->
  <finalName>springboot</finalName>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>

```

10.2.2. Starter 修改

添加容器启动加载文件 (类似于读取 web.xml)，这里通过继承 SpringBootServletInitializer 类并重写 configure 方法来实现，在部署项目时指定外部 Tomcat 读取项目入口方法。

```

@SpringBootApplication
public class Starter extends SpringBootServletInitializer {
    private static Logger logger = LoggerFactory.getLogger(Starter.class);

    public static void main(String[] args) {
        logger.info("SpringBoot 应用开始启动...");
        SpringApplication.run(Starter.class);
    }

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder
builder) {
        return builder.sources(Starter.class);
    }
}

```

10.2.3. 打包操作

执行命令，生成对应的 war 包

```

▼ target
  ► classes
  ► generated-sources
  ► maven-archiver
  ► maven-status
  ► springboot
  ► springboot.war
  ► springboot.war.original

```

10.2.4. 部署并访问

- 外部tomcat部署并访问

« 本地磁盘 (C:) » software » Tomcat » apache-tomcat-8.5.47 » webapps »

名称

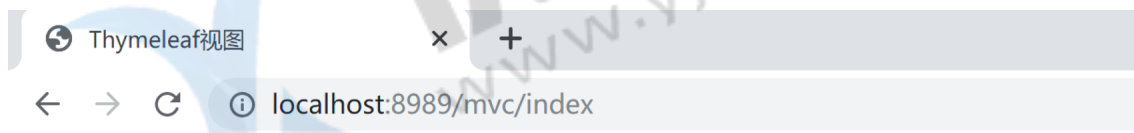
docs
examples
host-manager
manager
ROOT
springboot.war

```
C:\Windows\system32\cmd.exe
14-May-2020 17:30:19.706 信息 [main] org.apache.catalina.core.StandardService.startInternal Starting service [Catalina]
14-May-2020 17:30:19.709 信息 [main] org.apache.catalina.core.StandardEngine.startInternal Starting Servlet Engine: Apache Tomcat/8.5.47
14-May-2020 17:30:19.735 信息 [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployWAR Deploying web application archive [C:\software\Tomcat\
apache-tomcat-8.5.47\webapps\springboot.war]
LOGBACK: No context given for c.q.l.core.rolling.SizeAndTimeBasedRollingPolicy@2008143060

SpringBoot

17:30:22 [localhost-startStop-1] INFO com.xxxx.springboot.Starter- Starting Starter v1.0-SNAPSHOT on 3FDE with PID 15908 (C:\software\Tomcat\apache-tomcat-8
5.47\webapps\springboot\WEB-INF\classes started by ls in C:\software\Tomcat\apache-tomcat-8.5.47\bin)
17:30:22 [localhost-startStop-1] INFO com.xxxx.springboot.Starter- The following profiles are active: test
17:30:24 [localhost-startStop-1] INFO org.springframework.web.context.ContextLoader- Root WebApplicationContext: initialization completed in 1163 ms
17:30:24 [localhost-startStop-1] INFO org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor- Initializing ExecutorService 'applicationTaskExecuto
r'
17:30:25 [localhost-startStop-1] INFO org.springframework.boot.autoconfigure.web.servlet>WelcomePageHandlerMapping- Adding welcome page template: index
17:30:25 [localhost-startStop-1] INFO com.xxxx.springboot.Starter- Started Starter in 3.424 seconds (JVM running for 6.349)
14-May-2020 17:30:25.285 信息 [localhost-startStop-1] org.apache.jasper.servlet.TldScanner.scanJars 至少有一个JAR被扫描用于TLD但尚未包含TLD。 为此记录器启用
测试日志记录，以获取已扫描但未在其中找到TLD的完整JAR列表。 在扫描期间跳过不需要的JAR可以缩短启动时间和JSP编译时间。
14-May-2020 17:30:25.331 信息 [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployWAR Deployment of web application archive [C:\software\Tom
cat\apache-tomcat-8.5.47\webapps\springboot.war] has finished in [5,596] ms
14-May-2020 17:30:25.334 信息 [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory 把web 应用程序部署到目录 [C:\software\Tomcat\apa
che-tomcat-8.5.47\webapps\docs]
14-May-2020 17:30:25.370 信息 [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Deployment of web application directory [C:\soft
ware\Tomcat\apache-tomcat-8.5.47\webapps\docs] has finished in [36] ms
14-May-2020 17:30:25.371 信息 [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory 把web 应用程序部署到目录 [C:\software\Tomcat\apa
che-tomcat-8.5.47\webapps\examples]
14-May-2020 17:30:25.886 信息 [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Deployment of web application directory [C:\soft
ware\Tomcat\apache-tomcat-8.5.47\webapps\examples] has finished in [514] ms
14-May-2020 17:30:25.891 信息 [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory 把web 应用程序部署到目录 [C:\software\Tomcat\apa
che-tomcat-8.5.47\webapps\host-manager]
14-May-2020 17:30:25.946 信息 [localhost-startStop-1] org.apache.catalina.startup.HostConfig.deployDirectory Deployment of web application directory [C:\soft
ware\Tomcat\apache-tomcat-8.5.47\webapps\host-manager] has finished in [55] ms
```

- 浏览器访问



Hello SpringBoot